

Listado A.8: Proceso coche

```

1 #include <sys/types.h>
2 #include <unistd.h>
3 #include <stdlib.h>
4 #include <stdio.h>
5 #include <string.h>
6 #include <semaforoI.h>
7 #include <memoriaI.h>
8 #define MAX_TIME_CRUZAR 5
9
10 void coche (); void cruzar ();
11
12 int main (int argc, char *argv[]) {
13     coche(argv[1], argv[2], argv[3], argv[4]);
14     return 0;
15 }
16
17 /* salida permite identificar el origen del coche (N o S) */
18 void coche (char *salida, char *id_puente,
19             char *id_mutex, char *id_num_coches) {
20     int num_coches_handle, valor;
21     sem_t *puente, *mutex;
22
23     srand((int) getpid());
24
25     puente = get_sem(id_puente);
26     mutex = get_sem(id_mutex);
27     num_coches_handle = obtener_var(id_num_coches);
28
29     /* Acceso a la variable compartida 'num coches' */
30     wait_sem(mutex);
31
32     consultar_var(num_coches_handle, &valor);
33     modificar_var(num_coches_handle, ++valor);
34     /* Primer coche que intenta cruzar desde su extremo */
35     if (valor == 1)
36         wait_sem(puente); /* Espera el acceso al puente */
37
38     signal_sem(mutex);
39
40     cruzar(salida); /* Tardará un tiempo aleatorio */
41
42     wait_sem(mutex);
43
44     consultar_var(num_coches_handle, &valor);
45     modificar_var(num_coches_handle, --valor);
46     /* Ultimo coche en cruzar desde su extremo */
47     if (valor == 0)
48         signal_sem(puente); /* Libera el puente */
49
50     signal_sem(mutex);
51 }
52
53 void cruzar (char *salida) {
54     if (strcmp(salida, "N") == 0)
55         printf("%d cruzando de N a S...\n", getpid());
56     else
57         printf("%d cruzando de S a N...\n", getpid());
58     sleep(rand() % MAX_TIME_CRUZAR + 1);
59 }

```


B

Apéndice

Filósofos comensales

B.1. Enunciado

Los filósofos se encuentran comiendo o pensando. Todos comparten una mesa redonda con cinco sillas, una para cada filósofo. En el centro de la mesa hay una fuente de arroz y en la mesa sólo hay cinco palillos, de manera que cada filósofo tiene un palillo a su izquierda y otro a su derecha.

Cuando un filósofo piensa, entonces se abstrae del mundo y no se relaciona con ningún otro filósofo. Cuando tiene hambre, entonces intenta acceder a los palillos que tiene a su izquierda y a su derecha (necesita ambos). Naturalmente, un filósofo no puede quitarle un palillo a otro filósofo y sólo puede comer cuando ha cogido los dos palillos. Cuando un filósofo termina de comer, deja los palillos y se pone a pensar.

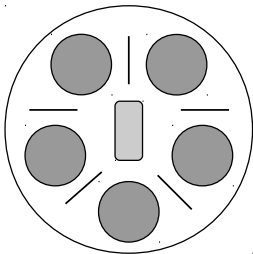


Figura B.1: Esquema gráfico del problema original de los filósofos comensales (*dining philosophers*).

B.2. Código fuente

Listado B.1: Makefile para la compilación automática

```
1 DIROBJ := obj/
2 DIREXE := exec/
3 DIRSRC := src/
4
5 CFLAGS := -c -Wall
6 LDLIBS := -lrt
7 CC := gcc
8
9 all : dirs manager filosofo
10
11 dirs:
12     mkdir -p $(DIROBJ) $(DIREXE)
13
14 manager: $(DIROBJ)manager.o
15     $(CC) -o $(DIREXE)$@ $^ $(LDLIBS)
16
17 filosofo: $(DIROBJ)filosofo.o
18     $(CC) -o $(DIREXE)$@ $^ $(LDLIBS)
19
20 $(DIROBJ)%.o: $(DIRSRC)%.c
21     $(CC) $(CFLAGS) $^ -o $@
22
23 clean :
24     rm -rf *~ core $(DIROBJ) $(DIREXE) $(DIRSRC)*~
```

Listado B.2: Archivo manager.c

```

1  #include <unistd.h>
2  #include <signal.h>
3  #include <string.h>
4  #include <stdlib.h>
5  #include <stdio.h>
6  #include <signal.h>
7  #include <wait.h>
8  #include <sys/types.h>
9  #include <sys/stat.h>
10 #include <mqueue.h>
11
12 #define BUZON_MESA "/buzon_mesa"
13 #define BUZON_PALILLO "/buzon_palillo_" /* Offset! */
14
15 #define FILOSOFOS 5
16
17 pid_t pids[FILOSOFOS];
18 mqd_t qHandlerMesa;
19 mqd_t qHandlerPalillos[FILOSOFOS];
20
21 void controlador (int senhal);
22 void liberarecursos ();
23 void finalizarprocesos ();
24
25 int main (int argc, char *argv[]) {
26     int i;
27     struct mq_attr mqAttr;
28     char buffer[64], caux[30], filosofo[1];
29     char buzon_palillo_izq[30], buzon_palillo_der[30];
30
31     // Reseteo del vector de pids.
32     memset (pids, 0, sizeof(pid_t) * (FILOSOFOS));
33
34     // Atributos del buzón mesa.
35     mqAttr.mq_maxmsg = (FILOSOFOS - 1); mqAttr.mq_msgsize = 64;
36
37     // Retrollamada de finalización.
38     if (signal(SIGINT, controlador) == SIG_ERR) {
39         fprintf(stderr, "Abrupt termination.\n");
40         exit(EXIT_FAILURE);
41     }
42
43     // Creación de buzones...
44     qHandlerMesa = mq_open(BUZON_MESA, O_WRONLY| O_CREAT,
45         S_IWUSR | S_IRUSR, &mqAttr);
46     for (i = 0; i < (FILOSOFOS - 1); i++)
47         // Para evitar el interbloqueo...
48         // Sólo 4 filósofos (máximo) intentan acceder a los palillos.
49         mq_send(qHandlerMesa, buffer, sizeof(buffer), 1);
50
51     // Para los buzones de los palillos.
52     mqAttr.mq_maxmsg = 1;
53
54     // Un buzón por palillo...
55     for (i = 0; i < FILOSOFOS; i++) {
56         sprintf(caux, "%s%d", BUZON_PALILLO, i);
57         qHandlerPalillos[i] = mq_open(caux, O_WRONLY | O_CREAT,
58             S_IWUSR | S_IRUSR, &mqAttr);
59         // Palillo inicialmente libre.
60         mq_send(qHandlerPalillos[i], buffer, sizeof(buffer), 0);
61     }
62
63     // Lanzamiento de procesos filósofo.
64     for (i = 0; i < FILOSOFOS ; i++)

```

```

65     if ((pids[i] = fork()) == 0) {
66         sprintf(filosofo, "%d", i);
67         sprintf (buzon_palillo_izq, "%s%d", BUZON_PALILLO, i);
68         sprintf (buzon_palillo_der, "%s%d",
69             BUZON_PALILLO, (i + 1) % FILOSOFOS);
70         execl("./exec/filosofo", "filosofo", filosofo, BUZON_MESA,
71             buzon_palillo_izq, buzon_palillo_der, NULL);
72     }
73
74     for (i = 0; i < FILOSOFOS; i++) waitpid(pids[i], 0, 0);
75     finalizarprocesos(); liberarecursos();
76     printf ("\n Fin del programa\n");
77     return 0;
78 }
79
80 void controlador (int senhal) {
81     finalizarprocesos(); liberarecursos();
82     printf ("\n Fin del programa (Control + C)\n");
83     exit(EXIT_SUCCESS);
84 }
85
86 void liberarecursos () {
87     int i; char caux[30];
88
89     printf ("\n Liberando buzones... \n");
90     mq_close(qHandlerMesa); mq_unlink(BUZON_MESA);
91
92     for (i = 0; i < FILOSOFOS; i++) {
93         sprintf (caux, "%s%d", BUZON_PALILLO, i);
94         mq_close(qHandlerPalillos[i]); mq_unlink(caux);
95     }
96 }
97
98 void finalizarprocesos () {
99     int i;
100     printf ("----- Terminando ----- \n");
101     for (i = 0; i < FILOSOFOS; i++) {
102         if (pids[i]) {
103             printf ("Finalizando proceso [%d]... ", pids[i]);
104             kill(pids[i], SIGINT); printf ("<Ok>\n");
105         }
106     }
107 }

```

Listado B.3: Archivo filosofo.c

```

1 #include <unistd.h>
2 #include <signal.h>
3 #include <stdlib.h>
4 #include <stdio.h>
5 #include <signal.h>
6 #include <mqueue.h>
7 #define MAX_TIME_PENSAR 7
8 #define MAX_TIME_COMER 5
9
10 void filosofo (char *filosofo, char *buzon_mesa,
11               char *buzon_palillo_izq, char *buzon_palillo_der);
12 void controlador (int senhal);
13
14 int main (int argc, char *argv[]) {
15     filosofo(argv[1], argv[2], argv[3], argv[4]);
16     return 0;
17 }
18
19 void filosofo (char *filosofo, char *buzon_mesa,
20               char *buzon_palillo_izq, char *buzon_palillo_der) {
21     mq_d_t qHandlerMesa, qHandlerIzq, qHandlerDer;
22     int n_filosofo;
23     char buffer[64];
24
25     // Retrollamada de finalización.
26     if (signal(SIGINT, controlador) == SIG_ERR) {
27         fprintf(stderr, "Abrupt termination.\n"); exit(EXIT_FAILURE);
28     }
29     n_filosofo = atoi(filosofo);
30
31     // Recupera buzones.
32     qHandlerMesa = mq_open(buzon_mesa, O_RDWR);
33     qHandlerIzq = mq_open(buzon_palillo_izq, O_RDWR);
34     qHandlerDer = mq_open(buzon_palillo_der, O_RDWR);
35
36     srand((int) getpid());
37     while (1) {
38         printf("[Filosofo %d] pensando...\n", n_filosofo);
39         sleep(rand() % MAX_TIME_PENSAR); // Pensar.
40
41         mq_receive(qHandlerMesa, buffer, sizeof(buffer), NULL);
42
43         // Hambriento (coger palillos)...
44         mq_receive(qHandlerIzq, buffer, sizeof(buffer), NULL);
45         mq_receive(qHandlerDer, buffer, sizeof(buffer), NULL);
46         // Comiendo...
47         printf("[Filosofo %d] comiendo...\n", n_filosofo);
48         sleep(rand() % MAX_TIME_COMER); // Comer.
49         // Dejar palillos...
50         mq_send(qHandlerIzq, buffer, sizeof(buffer), 0);
51         mq_send(qHandlerDer, buffer, sizeof(buffer), 0);
52
53         mq_send(qHandlerMesa, buffer, sizeof(buffer), 0);
54     }
55 }
56
57 void controlador (int senhal) {
58     printf("[Filosofo %d] Finalizado (SIGINT)\n", getpid());
59     exit(EXIT_SUCCESS);
60 }

```




Apéndice

La biblioteca de hilos de ICE

C.1. Fundamentos básicos

ICE (*Internet Communication Engine*) [6] es un *middleware* de comunicaciones orientado a objetos, es decir, ICE proporciona herramientas, *APIs*, y soporte de bibliotecas para construir aplicaciones distribuidas cliente-servidor orientadas a objetos.

Una aplicación ICE se puede usar en entornos heterogéneos. Los clientes y los servidores pueden escribirse en diferentes lenguajes de programación, pueden ejecutarse en distintos sistemas operativos y en distintas arquitecturas, y pueden comunicarse empleando diferentes tecnologías de red. La tabla C.1 resume las principales características de ICE.

Los principales **objetivos de diseño** de ICE son los siguientes:

- *Middleware* listo para usarse en sistemas heterogéneos.
- Provee un conjunto completo de características que soporten el desarrollo de aplicaciones distribuidas reales en un amplio rango de dominios.
- Es fácil de aprender y de usar.
- Proporciona una implementación eficiente en ancho de banda, uso de memoria y CPU.



Figura C.1: ZeroC ICE es un *middleware* de comunicaciones que se basa en la filosofía de CORBA pero con una mayor simplicidad y practicidad.

Nombre	Internet Communications Engine (ICE)
Definido por	ZeroC Inc. (http://www.zeroc.com)
Documentación	http://doc.zeroc.com/display/Doc/Home
Lenguajes	C++, Java, C#, Visual Basic, Python, PHP, Ruby
Plataformas	Windows, Windows CE, Unix, GNU/Linux, *BSD OSX, Symbian OS, J2RE 1.4 o superior, J2ME
Destacado	APIs claras y bien diseñadas Conjunto de servicios muy cohesionados Despliegue, persistencia, cifrado...
Descargas	http://zeroc.com/download.html

Tabla C.1: ZeroC ICE. Resumen de características.

- Implementación basada en la seguridad.

Para instalar ICE en sistemas operativos Debian GNU/Linux, ejecute los siguientes comandos:

```
$ sudo apt-get update
$ sudo apt-get install zeroc-ice34
```

C.2. Manejo de hilos

ICE proporciona distintas utilidades para la gestión de la concurrencia y el manejo de hilos. Respecto a este último aspecto, ICE proporciona una **abstracción** muy sencilla para el manejo de hilos con el objetivo de explotar el paralelismo mediante la creación de hilos dedicados. Por ejemplo, sería posible crear un hilo específico que atienda las peticiones de una interfaz gráfica o crear una serie de hilos encargados de tratar con aquellas operaciones que requieren una gran cantidad de tiempo, ejecutándolas en segundo plano.

En este contexto, ICE proporciona una abstracción de hilo muy sencilla que posibilita el desarrollo de aplicaciones multihilo altamente portables e independientes de la plataforma de hilos nativa. Esta abstracción está representada por la clase **IceUtil::Thread**.



ICE maneja las peticiones a los servidores mediante un *pool* de hilos con el objetivo de incrementar el rendimiento de la aplicación. El desarrollador es el responsable de gestionar el acceso concurrente a los datos.

Como se puede apreciar en el siguiente listado de código, *Thread* es una clase abstracta con una función virtual pura denominada **run()**. El desarrollador ha de implementar esta función para poder crear un hilo, de manera que *run()* se convierta en el punto de inicio de la ejecución de dicho hilo. Note que no es posible arrojar excepciones desde

esta función. El núcleo de ejecución de ICE instala un manejador de excepciones que llama a la función `::std::terminate()` si se arroja alguna excepción.

Listado C.1: La clase `IceUtil::Thread`

```

1 class Thread :virtual public Shared {
2 public:
3
4 // Función a implementar por el desarrollador.
5 virtual void run () = 0;
6
7 ThreadControl start (size_t stBytes = 0);
8 ThreadControl start (size_t stBytes, int priority);
9 ThreadControl getThreadControl () const;
10 bool isAlive () const;
11
12 bool operator== (const Thread&) const;
13 bool operator!= (const Thread&) const;
14 bool operator< (const Thread&) const;
15 };
16
17 typedef Handle<Thread> ThreadPtr;
```

El resto de funciones de *Thread* son las siguientes:

- **start()**, cuya responsabilidad es arrancar al hilo y llamar a la función *run()*. Es posible especificar el tamaño en *bytes* de la pila del nuevo hilo, así como un valor de prioridad. El valor de retorno de *start()* es un objeto de tipo *ThreadControl*, el cual se discutirá más adelante.
- **getThreadControl()**, que devuelve el objeto de la clase *ThreadControl* asociado al hilo.
- **id()**, que devuelve el identificador único asociado al hilo. Este valor dependerá del soporte de hilos nativo (por ejemplo, POSIX *pthreads*).
- **isAlive()**, que permite conocer si un hilo está en ejecución, es decir, si ya se llamó a *start()* y la función *run()* no terminó de ejecutarse.
- La **sobrecarga de los operadores** de comparación permiten hacer uso de hilos en contenedores de STL que mantienen relaciones de orden.

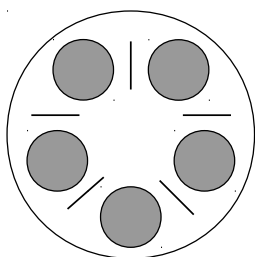


Figura C.2: Abstracción gráfica del problema de los filósofos comensales, donde cinco filósofos piensan y comparten cinco palillos para comer.

Para mostrar cómo llevar a cabo la implementación de un hilo específico a partir de la biblioteca de ICE se usará como ejemplo uno de los problemas clásicos de sincronización: el problema de los **filósofos comensales**, discutido en la sección 2.3.3.

Básicamente, los filósofos se encuentran comiendo o pensando. Todos comparten una mesa redonda con cinco sillas, una para cada filósofo. Cada filósofo tiene un plato individual con arroz y en la mesa sólo hay cinco palillos, de manera que cada filósofo tiene un palillo a su izquierda y otro a su derecha.

Cuando un filósofo piensa, entonces se abstrae del mundo y no se relaciona con ningún otro filósofo. Cuando tiene hambre, entonces intenta coger a los palillos que tiene a su izquierda y a su derecha (necesita ambos). Naturalmente, un filósofo no puede quitarle un palillo a otro filósofo y sólo puede comer cuando ha cogido los dos palillos. Cuando un filósofo termina de comer, deja los palillos y se pone a pensar.

La solución que se discutirá en esta sección se basa en implementar el filósofo como un hilo independiente. Para ello, se crea la clase *FilosofoThread* que se expone a continuación.

Listado C.2: La clase FilosofoThread

```
1  #ifndef __FILOSOFO__
2  #define __FILOSOFO__
3
4  #include <iostream>
5  #include <IceUtil/Thread.h>
6  #include <Palillo.h>
7
8  #define MAX_COMER 3
9  #define MAX_PENSAR 7
10
11 using namespace std;
12
13 class FilosofoThread : public IceUtil::Thread {
14
15 public:
16     FilosofoThread (const int& id, Palillo* izq, Palillo *der);
17
18     virtual void run ();
19
20 private:
21     void coger_palillos ();
22     void dejar_palillos ();
23     void comer () const;
24     void pensar () const;
25
26     int _id;
27     Palillo *_pIzq, *_pDer;
28 };
29
30 #endif
```

La implementación de la función *run()* es trivial a partir de la descripción del enunciado del problema.

Listado C.3: La función `FilosofoThread::run()`

```
1 void
2 FilosofoThread::run ()
3 {
4     while (true) {
5         coger_palillos();
6         comer();
7         dejar_palillos();
8         pensar();
9     }
10 }
```

El problema de concurrencia viene determinado por el acceso de los filósofos a los palillos, los cuales representan la sección crítica asociada a cada uno de los hilos que implementan la vida de un filósofo. En otras palabras, es necesario establecer algún tipo de mecanismo de sincronización para garantizar que dos filósofos no cogen un mismo palillo de manera simultánea. Antes de abordar esta problemática, se mostrará cómo lanzar los hilos que representan a los cinco filósofos.



El problema de los filósofos comensales es uno de los problemas clásicos de sincronización y existen muchas modificaciones del mismo que permiten asimilar mejor los conceptos teóricos de la programación concurrente.

El siguiente listado muestra el código básico necesario para lanzar los filósofos (hilos).

Note cómo en la línea [23] se llama a la función *run()* de *Thread* para comenzar la ejecución del mismo. Los objetos de tipo *ThreadControl* devueltos se almacenan en un vector para, posteriormente, *unir* los hilos creados. Para ello, se hace uso de la función *join()* de la clase *ThreadControl*, tal y como se muestra en la línea [29].

Listado C.4: Creación y control de hilos con ICE

```

1 #include <IceUtil/Thread.h>
2 #include <vector>
3 #include <Palillo.h>
4 #include <Filosofo.h>
5
6 #define NUM 5
7
8 int main (int argc, char *argv[]) {
9     std::vector<Palillo*> palillos;
10    std::vector<IceUtil::ThreadControl> threads;
11    int i;
12
13    // Se instancian los palillos.
14    for (i = 0; i < NUM; i++)
15        palillos.push_back(new Palillo);
16
17    // Se instancian los filósofos.
18    for (i = 0; i < NUM; i++) {
19        // Cada filósofo conoce los palillos
20        // que tiene a su izda y derecha.
21        IceUtil::ThreadPtr t =
22            new FilosofoThread(i, palillos[i], palillos[(i + 1) % NUM]);
23        // start sobre hilo devuelve un objeto ThreadControl.
24        threads.push_back(t->start());
25    }
26
27    // 'Unión' de los hilos creados.
28    std::vector<IceUtil::ThreadControl>::iterator it;
29    for (it = threads.begin(); it != threads.end(); ++it) it->join();
30
31    return 0;
32 }

```

C.3. Exclusión mutua básica

El problema de los filósofos plantea la necesidad de algún tipo de mecanismo de sincronización básico para garantizar el acceso exclusivo sobre cada uno de los palillos. La opción más directa consiste en asociar un **cerrojo** a cada uno de los palillos de manera individual. Así, si un filósofo intenta coger un palillo que está libre, entonces lo cogerá adquiriendo el cerrojo, es decir, *cerrándolo*. Si el palillo está ocupado, entonces el filósofo esperará hasta que esté libre.



Típicamente, las operaciones para adquirir y liberar un cerrojo se denominan *lock()* y *unlock()*, respectivamente. La metáfora de cerrojo representa perfectamente la adquisición y liberación de recursos compartidos.

ICE proporciona la clase **Mutex** para modelar esta problemática de una forma sencilla y directa.

Listado C.5: La clase `IceUtil::Mutex`

```

1 class Mutex {
2 public:
3     Mutex ();
4     Mutex (MutexProtocol p);
5     ~Mutex ();
6
7     void lock      () const;
8     bool tryLock   () const;
9     void unlock    () const;
10
11     typedef LockT<Mutex> Lock;
12     typedef TryLockT<Mutex> TryLock;
13 };

```

Las funciones miembro más importantes de esta clase son las que permiten adquirir y liberar el cerrojo:

- **lock()**, que intenta adquirir el cerrojo. Si éste ya estaba cerrado, entonces el hilo que invocó a la función se suspende hasta que el cerrojo quede libre. La llamada a dicha función retorna cuando el hilo ha adquirido el cerrojo.
- **tryLock()**, que intenta adquirir el cerrojo. A diferencia de *lock()*, si el cerrojo está cerrado, la función devuelve *false*. En caso contrario, devuelve *true* con el cerrojo cerrado.
- **unlock()**, que libera el cerrojo.

Es importante considerar que la clase *Mutex* proporciona un mecanismo de exclusión mutua básico y no recursivo, es decir, no se debe llamar a *lock()* más de una vez desde un hilo, ya que esto provocará un comportamiento inesperado. Del mismo modo, no se debería llamar a *unlock()* a menos que un hilo haya adquirido previamente el cerrojo mediante *lock()*. En la siguiente sección se estudiará otro mecanismo de sincronización que mantiene una semántica recursiva.

La clase *Mutex* se puede utilizar para gestionar el acceso concurrente a los palillos. En la solución planteada a continuación, un palillo es simplemente una especialización de *IceUtil::Mutex* con el objetivo de incrementar la semántica de dicha solución.

Listado C.6: La clase `Palillo`

```

1 #ifndef __PALILLO__
2 #define __PALILLO__
3
4 #include <IceUtil/Mutex.h>
5
6 class Palillo : public IceUtil::Mutex {
7 };
8
9 #endif

```

Para que los filósofos puedan utilizar los palillos, habrá que utilizar la funcionalidad previamente discutida, es decir, las funciones *lock()* y

unlock(), en las funciones *coger_palillos()* y *dejar_palillos()*. La solución planteada garantiza que no se ejecutarán dos llamadas a *lock()* sobre un palillo por parte de un mismo hilo, ni tampoco una llamada sobre *unlock()* si previamente no se adquirió el palillo.

Listado C.7: Acceso concurrente a los palillos

```
1 void
2 FilosofoThread::coger_palillos ()
3 {
4     _pIzq->lock();
5     _pDer->lock();
6 }
7
8 void
9 FilosofoThread::dejar_palillos ()
10 {
11     _pIzq->unlock();
12     _pDer->unlock();
13 }
```

La solución planteada es poco flexible debido a que los filósofos están inactivos durante el periodo de tiempo que pasa desde que dejan de pensar hasta que cogen los dos palillos. Una posible variación a la solución planteada hubiera sido continuar pensando (al menos hasta un número máximo de ocasiones) si los palillos están ocupados. En esta variación se podría utilizar la función *tryLock()* para modelar dicha problemática.



Si todos los filósofos cogen al mismo tiempo el palillo que está a su izquierda se producirá un interbloqueo ya que la solución planteada no podría avanzar hasta que un filósofo coja ambos palillos.

C.3.1. Evitando interbloqueos

El uso de las funciones *lock()* y *unlock()* puede generar problemas importantes si, por alguna situación no controlada, un cerrojo previamente adquirido con *lock()* no se libera posteriormente con una llamada a *unlock()*. El siguiente listado de código muestra esta problemática.

En la línea [8], la sentencia *return* implica abandonar *mi_funcion()* sin haber liberado el cerrojo previamente adquirido en la línea [6]. En este contexto, es bastante probable que se genere una potencial situación de interbloqueo que paralizaría la ejecución del programa. La generación de excepciones no controladas representa otro caso representativo de esta problemática.



Aunque existen diversos esquemas para evitar y recuperarse de un interbloqueo, los sistemas operativos modernos suelen optar por asumir que nunca se producirán, delegando en el programador la implementación de soluciones seguras.

Listado C.8: Potencial interbloqueo

```
1 #include <IceUtil/Mutex.h>
2
3 class Test {
4 public:
5     void mi_funcion () {
6         _mutex.lock();
7         for (int i = 0; i < 5; i++)
8             if (i == 3) return; // Generará un problema...
9         _mutex.unlock();
10    }
11
12 private:
13     IceUtil::Mutex _mutex;
14 };
15
16 int main (int argc, char *argv[]) {
17     Test t;
18     t.mi_funcion();
19     return 0;
20 }
```

Para evitar este tipo de problemas, la clase *Mutex* proporciona las definiciones de tipo *Lock* y *TryLock*, que representan plantillas muy sencillas compuestas de un constructor en el que se llama a *lock()* y *tryLock()*, respectivamente. En el destructor se llama a *unlock()* si el cerrojo fue previamente adquirido cuando la plantilla quede fuera de ámbito. En el ejemplo anterior, sería posible garantizar la liberación del cerrojo al ejecutar *return*, ya que quedaría fuera del alcance de la función. El siguiente listado de código muestra la modificación realizada para evitar interbloqueos.



Es recomendable usar siempre *Lock* y *TryLock* en lugar de las funciones *lock()* y *unlock* para facilitar el entendimiento y la mantenibilidad del código.

Listado C.9: Evitando interbloqueos con Lock

```

1 #include <IceUtil/Mutex.h>
2
3 class Test {
4 public:
5     void mi_funcion () {
6         IceUtil::Mutex::Lock lock(_mutex);
7         for (int i = 0; i < 5; i++)
8             if (i == 3) return; // Ningún problema...
9     } // El destructor de lock libera el cerrojo.
10
11 private:
12     IceUtil::Mutex _mutex;
13 };

```



No olvide liberar un cerrojo sólo si fue previamente adquirido y llamar a *unlock()* tantas veces como a *lock()* para que el cerrojo quede disponible para otro hilo.

C.3.2. Flexibilizando el concepto de *mutex*

Además de proporcionar cerrojos con una semántica no recursiva, ICE también proporciona la clase *IceUtil::RecMutex* con el objetivo de que el desarrollador pueda manejar cerrojos *recursivos*. La interfaz de esta nueva clase es exactamente igual que la clase *IceUtil::Mutex*.

Listado C.10: La clase *IceUtil::RecMutex*

```

1 class RecMutex {
2 public:
3     RecMutex ();
4     RecMutex (MutexProtocol p);
5     ~RecMutex ();
6
7     void lock () const;
8     bool tryLock () const;
9     void unlock () const;
10
11     typedef LockT<RecMutex> Lock;
12     typedef TryLockT<RecMutex> TryLock;
13 };

```

Sin embargo, existe una diferencia fundamental entre ambas. Internamente, el cerrojo recursivo está implementado con un contador inicializado a cero. Cada llamada a *lock()* incrementa el contador, mientras que cada llamada a *unlock()* lo decrementa. El cerrojo estará disponible para otro hilo cuando el contador alcance el valor de cero.

C.4. Introduciendo monitores

Tanto la clase *Mutex* como la clase *MutexRec* son mecanismos de sincronización básicos que permiten que sólo un hilo esté activo, en un instante de tiempo, dentro de la sección crítica. En otras palabras, para que un hilo pueda acceder a la sección crítica, otro ha de abandonarla. Esto implica que, cuando se usan cerrojos, no resulta posible suspender un hilo dentro de la sección crítica para, posteriormente, despertarlo cuando se cumpla una determinada condición.



Las soluciones de alto nivel permiten que el desarrollador tenga más flexibilidad a la hora de solucionar un problema. Este planteamiento se aplica perfectamente al uso de monitores.

Para tratar este tipo de problemas, la biblioteca de hilos de ICE proporciona la **clase Monitor**. En esencia, un monitor es un mecanismo de sincronización de más alto nivel que, al igual que un cerrojo, protege la sección crítica y garantiza que solamente pueda existir un hilo activo dentro de la misma. Sin embargo, un monitor permite suspender un hilo dentro de la sección crítica posibilitando que otro hilo pueda acceder a la misma. Este segundo hilo puede abandonar el monitor, liberándolo, o suspenderse dentro del monitor. De cualquier modo, el hilo original se despierta y continua su ejecución dentro del monitor. Este esquema es escalable a múltiples hilos, es decir, varios hilos pueden suspenderse dentro de un monitor.

Desde un punto de vista general, los monitores proporcionan un mecanismo de sincronización más flexible que los cerrojos, ya que es posible que un hilo compruebe una condición y, si ésta es falsa, el hilo se pause. Si otro hilo cambia dicha condición, entonces el hilo original continúa su ejecución.

El siguiente listado de código muestra la declaración de la clase **IceUtil::Monitor**. Note que se trata de una clase que hace uso de plantillas y requiere como parámetro bien *Mutex* o *RecMutex*, en función de si el monitor mantendrá una semántica no recursiva o recursiva.

Las funciones miembro de esta clase son las siguientes:

- **lock()**, que intenta adquirir el monitor. Si éste ya estaba cerrado, entonces el hilo que la invoca se suspende hasta que el monitor quede disponible. La llamada retorna con el monitor cerrado.
- **tryLock()**, que intenta adquirir el monitor. Si está disponible, la llamada devuelve *true* con el monitor cerrado. Si éste ya estaba cerrado antes de relizar la llamada, la función devuelve *false*.
- **unlock()**, que libera el monitor. Si existen hilos bloqueados, entonces uno de ellos se despertará y cerrará de nuevo el monitor.

Listado C.11: La clase IceUtil::Monitor

```

1  template <class T>
2  class Monitor {
3  public:
4      void lock      () const;
5      void unlock   () const;
6      bool tryLock  () const;
7
8      void wait      () const;
9      bool timedWait (const Time&) const;
10     void notify    ();
11     void notifyAll  ();
12
13     typedef LockT<Monitor<T> > Lock;
14     typedef TryLockT<Monitor<T> > TryLock;
15 };

```

- **wait()**, que suspende al hilo que invoca a la función y, de manera simultánea, libera el monitor. Un hilo suspendido por *wait()* se puede despertar por otro hilo que invoque a la función *notify()* o *notifyAll()*. Cuando la llamada retorna, el hilo suspendido continúa su ejecución con el monitor cerrado.
- **timedWait()**, que suspende al hilo que invoca a la función hasta un tiempo especificado por parámetro. Si otro hilo invoca a *notify()* o *notifyAll()* despertando al hilo suspendido antes de que el *timeout* expire, la función devuelve *true* y el hilo suspendido resume su ejecución con el monitor cerrado. En otro caso, es decir, si el *timeout* expira, *timedWait()* devuelve *false*.
- **notify()**, que despierta a un único hilo suspendido debido a una invocación sobre *wait()* o *timedWait()*. Si no existiera ningún hilo suspendido, entonces la invocación sobre *notify()* se *pierde*. Llevar a cabo una notificación no implica que otro hilo reanude su ejecución inmediatamente. En realidad, esto ocurriría cuando el hilo que invoca a *wait()* o *timedWait()* o libera el monitor.
- **notifyAll()**, que despierta a todos los hilos suspendidos por *wait()* o *timedWait()*. El resto del comportamiento derivado de invocar a esta función es idéntico a *notify()*.



No olvide comprobar la condición asociada al uso de *wait* siempre que se retorne de una llamada a la misma.

C.4.1. Ejemplo de uso de monitores

Imagine que desea modelar una situación en un videojuego en la que se controlen los **recursos disponibles** para acometer una determinada tarea. Estos recursos se pueden manipular desde el punto de

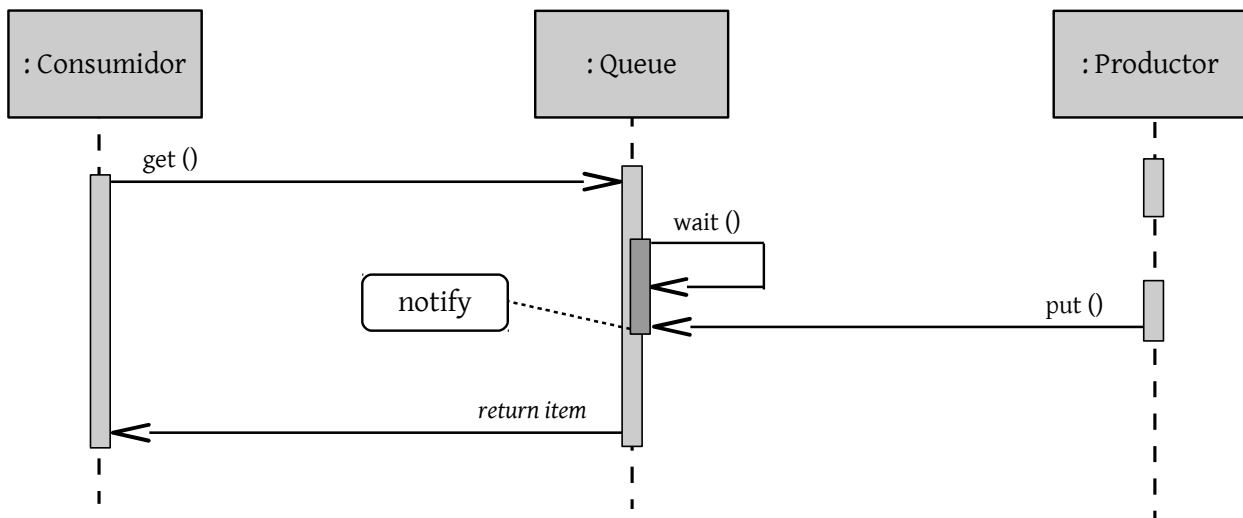


Figura C.3: Representación gráfica del uso de un monitor.

vista de la generación o producción y desde el punto de vista de la destrucción o consumición. En otras palabras, el problema clásico del productor/consumidor.

Por ejemplo, imagine un juego de acción de tercera persona en el que el personaje es capaz de **acumular slots** para desplegar algún tipo de poder especial. Inicialmente, el personaje tiene los *slots* vacíos pero, conforme el juego evoluciona, dichos *slots* se pueden rellenar atendiendo a varios criterios independientes. Por ejemplo, si el jugador acumula una cantidad determinada de puntos, entonces obtendría un *slot*. Si el personaje principal vence a un enemigo con un alto nivel, entonces obtendría otro *slot*. Por otra parte, el jugador podría hacer uso de estas habilidades especiales cuando así lo considere, siempre y cuando tenga al menos un *slot* relleno.

Este supuesto plantea la problemática de sincronizar el acceso concurrente a dichos *slots*, tanto para su consumo como para su generación. Además, hay que tener en cuenta que sólo será posible consumir un *slot* cuando haya al menos uno disponible. Es decir, la problemática discutida también plantea ciertas restricciones o condiciones que se han de satisfacer para que el jugador pueda lanzar una habilidad especial.

En este contexto, el uso de los monitores proporciona gran **flexibilidad** para modelar una solución a este supuesto. El siguiente listado de código muestra una posible implementación de la estructura de datos que podría dar soporte a la solución planteada, haciendo uso de los monitores de la biblioteca de hilos de ICE.

Listado C.12: Utilizando monitores

```

1 #include <IceUtil/Monitor.h>
2 #include <IceUtil/Mutex.h>
3 #include <deque>
4
5 using namespace std;
6
7 template<class T>
8 class Queue : public IceUtil::Monitor<IceUtil::Mutex> {
9 public:
10 void put (const T& item) { // Añade un nuevo item.
11     IceUtil::Monitor<IceUtil::Mutex>::Lock lock(*this);
12     _queue.push_back(item);
13     notify();
14 }
15 T get () { // Consume un item.
16     IceUtil::Monitor<IceUtil::Mutex>::Lock lock(*this);
17     while (_queue.size() == 0)
18         wait();
19     T item = _queue.front();
20     _queue.pop_front();
21     return item;
22 }
23
24 private:
25     deque<T> _queue; // Cola de doble entrada.
26 };

```

Como se puede apreciar, la clase definida es un tipo particular de monitor sin semántica recursiva, es decir, definido a partir de *IceUtil::Mutex*. Dicha clase tiene como variable miembro una cola de doble entrada que maneja tipos de datos genéricos, ya que la clase definida hace uso de una plantilla. Además, esta clase proporciona las dos operaciones típicas de *put()* y *get()* para añadir y obtener elementos.

Hay, sin embargo, dos características importantes a destacar en el **diseño** de esta estructura de datos:

1. El acceso concurrente a la variable miembro de la clase se controla mediante el propio monitor, haciendo uso de la función *lock()* (líneas [11](#) y [16](#)).
2. Si la estructura no contiene elementos, entonces el hilo se suspende mediante *wait()* (líneas [17-18](#)) hasta que otro hilo que ejecute *put()* realice la invocación sobre *notify()* (línea [13](#)).



No olvide... usar *Lock* y *TryLock* para evitar posibles interbloqueos causados por la generación de alguna excepción o una terminación de la función no prevista inicialmente.

Recuerde que para que un hilo bloqueado por *wait()* reanude su ejecución, otro hilo ha de ejecutar *notify()* y liberar el monitor mediante *unlock()*. Sin embargo, en el anterior listado de código no existe ninguna llamada explícita a *unlock()*. ¿Es incorrecta la solución? La

respuesta es no, ya que la liberación del monitor se delega en *Lock* cuando la función *put()* finaliza, es decir, justo después de ejecutar la operación *notify()* en este caso particular.

Volviendo al ejemplo anterior, considere dos hilos distintos que interactúan con la estructura creada, de manera genérica, para almacenar los *slots* que permitirán la activación de habilidades especiales por parte del jugador virtual. Por ejemplo, el **hilo asociado al productor** podría implementarse de la siguiente forma:

Suponiendo que el código del consumidor sigue la misma estructura, pero extrayendo elementos de la estructura de datos compartida, entonces sería posible lanzar distintos hilos para comprobar que el acceso concurrente sobre los distintos *slots* se realiza de manera adecuada. Además, sería sencillo visualizar, mediante mensajes por la salida estándar, que efectivamente los hilos consumidores se suspenden en *wait()* hasta que hay al menos algún elemento en la estructura de datos compartida con los productores.

Listado C.13: Hilo productor de slots

```
1 class Productor : public IceUtil::Thread {
2 public:
3     Productor (Queue<string> *_q):
4         _queue(_q) {}
5
6     void run () {
7         for (int i = 0; i < 5; i++) {
8             IceUtil::ThreadControl::sleep
9             (IceUtil::Time::seconds(rand() % 7));
10            _queue->put ("TestSlot");
11        }
12    }
13
14 private:
15     Queue<string> *_queue;
16 };
```

A continuación, resulta importante volver a discutir la solución planteada inicialmente para el manejo de monitores. En particular, la implementación de las funciones miembro *put()* y *get()* puede generar **sobrecarga** debido a que, cada vez que se añade un nuevo elemento a la estructura de datos, se realiza una invocación. Si no existen hilos esperando, la notificación se pierde. Aunque este hecho no conlleva ningún efecto no deseado, puede generar una reducción del rendimiento si el número de notificaciones se dispara.

Una posible solución a este problema consiste en llevar un **control explícito** de la existencia de consumidores de información, es decir, de hilos que invoquen a la función *get()*. Para ello, se puede incluir una variable miembro en la clase *Queue*, planteando un esquema mucho más eficiente.



El uso de alguna estructura de datos adicional puede facilitar el diseño de una solución. Este tipo de planteamientos puede incrementar la eficiencia de la solución planteada aunque haya una mínima sobrecarga.

Como se puede apreciar en el siguiente listado, el hilo productor sólo llevará a cabo una notificación en el caso de que haya algún hilo consumidor en espera. Para ello, consulta el valor de la variable miembro `_consumidoresEsperando`. Por otra parte, los hilos consumidores, es decir, los que invoquen a la función `get()` incrementan dicha variable antes de realizar un `wait()`, decrementándola cuando se despierten.

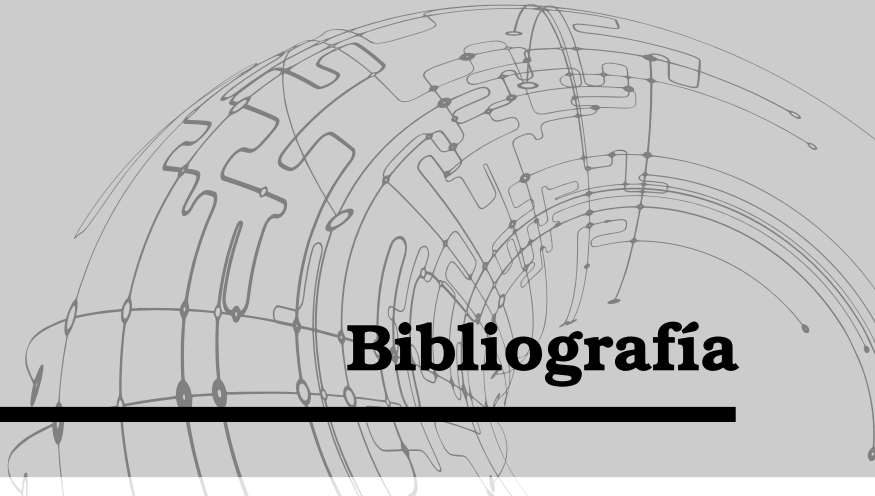
Listado C.14: Utilizando monitores (II)

```

1  #include <IceUtil/Monitor.h>
2  #include <IceUtil/Mutex.h>
3  #include <deque>
4
5  template<class T>
6  class Queue : public IceUtil::Monitor<IceUtil::Mutex> {
7  public:
8      Queue () : _consumidoresEsperando(0) {}
9
10     void put (const T& item) { // Añade un nuevo item.
11         IceUtil::Monitor<IceUtil::Mutex>::Lock lock(*this);
12         _queue.push_back(item);
13         if (_consumidoresEsperando)
14             notify();
15     }
16
17     T get () { // Consume un item.
18         IceUtil::Monitor<IceUtil::Mutex>::Lock lock(*this);
19         while (_queue.size() == 0) {
20             try {
21                 _consumidoresEsperando++;
22                 wait();
23                 _consumidoresEsperando--;
24             }
25             catch (...) {
26                 _consumidoresEsperando--;
27                 throw;
28             }
29         }
30         T item = _queue.front();
31         _queue.pop_front();
32         return item;
33     }
34
35 private:
36     std::deque<T> _queue; // Cola de doble entrada.
37     int _consumidoresEsperando;
38 };

```

Note que el acceso a la variable miembro `_consumidoresEsperando` es exclusivo y se garantiza gracias a la adquisición del monitor justo al ejecutar la operación de generación o consumo de información por parte de algún hilo.



Bibliografía

- [1] G.R. Andrews. *Concurrent programming: principles and practice*. Benjamin/Cummings Publishing Company, 1991.
- [2] J.G.P. Barnes and J.G.P. Barnes. *Programming in ADA*. Cambridge Univ Press, 1989.
- [3] A. Burns and A. Wellings. *Sistemas de Tiempo Real y Lenguajes de Programación (3ª Edición)*. Addison-Wesley, 2005.
- [4] D. Cornhill, L. Sha, and J.P. Lehoczky. Limitations of ada for real-time scheduling. In *ACM SIGAda Ada Letters*, volume 7, pages 33–39. ACM, 1987.
- [5] A.B. Downey. *The Little Book of Semaphores (version 2.1.5)*. Green Tea Press, 2008.
- [6] M. Henning. A new approach to object-oriented middleware. *IEEE Internet Computing*, 8(1):66–75, 2004.
- [7] C.L. Liu and J.W. Layland. Scheduling algorithms for multiprogramming in a hard-real-time environment. *Journal of the ACM (JACM)*, 20(1):46–61, 1973.
- [8] K.A. Robbins and S. Robbins. *Unix Programación Práctica, Guía para la Concurrencia, la Comunicación y los Multihilos*. Prentice Hall, 1997.
- [9] M.J. Rochkind. *Advanced Unix Programming (2nd Edition)*. Addison-Wesley, 2004.
- [10] H. Schildt. *C++ from the Ground Up (3rd Edition)*. McGraw-Hill Osborne, 2003.
- [11] L. Sha, R. Rajkumar, and J.P. Lehoczky. Priority inheritance protocols: An approach to real-time synchronization. *Computers, IEEE Transactions on*, 39(9):1175–1185, 1990.
- [12] A. Silberschatz, P.B. Galvin, and G. Gagne. *Fundamentos de Sistemas Operativos (7ª Edición)*. McGraw Hill, 2006.

- [13] W. Stallings. *Sistemas Operativos. Aspectos internos y principios de diseño*. Prentice Hall, 2006.
- [14] B. Stroustrup. *El lenguaje de programación C++*. Pearson Education, 2001.
- [15] A.S. Tanenbaum. *Sistemas Operativos Modernos (3ª Edición)*. Pearson, 2009.
- [16] P. Wegner and S.A. Smolka. Processes, tasks, and monitors: a comparative study of concurrent programming primitives. *Software Engineering, IEEE Transactions on*, 9(4):446–462, 1983.

Este libro fue maquetado mediante el sistema de composición de
textos $\text{\LaTeX}$ utilizando software del proyecto GNU.

Ciudad Real, a 8 de Enero de 2014.

